

UNIVERSITY OF WASHINGTON

TINY MACHINE LEARNING FOR ULTRA LOW-POWER EDGE COMPUTING

EE P 564 A

STOP & LOG:
A Voice-Triggered Distraction Tracker

Authors: Eric Peng, William Liao

May 01, 2026



Project Proposal

1. Problem Statement

Distraction is a widespread challenge that affects productivity and self-awareness. Many people struggle to recognize how often they lose focus during work or study sessions, making it difficult to identify patterns and improve their habits.

This project addresses that problem with a lightweight, always-on voice-triggered distraction logger. When a user notices they have become distracted, they say the keyword “STOP”. The Arduino Nano 33 BLE Sense detects this keyword in real time using a TinyML model running on-device. Upon detection, a timestamp is logged to a CSV file on the user’s computer via the Serial connection. Over time, this log reveals how frequently and at what times of day the user tends to get distracted, enabling meaningful self-assessment.

This problem is a natural fit for TinyML because the solution requires:

- Always-on keyword detection with low power consumption
- On-device inference with no dependency on cloud or internet connectivity
- Real-time response to a single spoken word
- Deployment on a resource-constrained microcontroller (256 KB RAM, 1 MB Flash)

2. Dataset(s)

Primary training dataset: Google Speech Commands Dataset v0.02.

- Source: https://storage.googleapis.com/download.tensorflow.org/data/speech_commands_v0.02.tar.gz
- Over 100,000 one-second WAV audio clips at 16 kHz
- Covers 35 spoken words including “stop”, “yes”, “no”, “go”
- Includes background noise and silence samples for robust training

Key features used:

- Target keyword: “stop” (already in dataset)
- Negative class: background noise and other words
- Sample rate: 16,000 Hz
- Clip duration: 1 second
- Audio features: MFCC (Mel-Frequency Cepstral Coefficients)

Optional supplementary recordings: To improve accuracy for the user’s own voice and environment, additional personal recordings of “stop” (about 50 to 100 samples) can be collected using the Lab 4 Colab audio recorder and merged with the Speech Commands dataset.

3. Modeling Approach

Sensor:

- Microphone (built-in on Arduino Nano 33 BLE Sense)

Audio preprocessing pipeline (following Lab 4):

1. Raw audio captured at 16 kHz
2. Short-Time Fourier Transform (STFT) to spectrogram
3. Mel-filter bank applied to create Mel spectrogram
4. Log compression to MFCC features (40 coefficients)

Neural network architecture:

- Type: Depthwise Separable Convolutional Neural Network (DS-CNN)
- Input: MFCC feature matrix (49 x 40)

- Output: 2 classes: stop and background/unknown

Compression for deployment:

- Post-training quantization from 32-bit float to 8-bit integers using TensorFlow Lite
- Target model size: under 20 KB to fit Arduino Nano 33 BLE Sense Flash
- Quantization reference data: subset of Speech Commands dataset (same as Lab 4)

4. Deployment Plan

Hardware:

- Board: Arduino Nano 33 BLE Sense
- Sensors used: built-in MP34DT05 PDM microphone
- No additional hardware required

Deployment steps:

1. Train and quantize model in Google Colab, export as TensorFlow Lite .cc file
2. Replace `micro_features_model.cpp` in the `micro_speech` Arduino sketch with the new model
3. Upload the sketch to Arduino Nano 33 BLE Sense via Arduino IDE

Runtime behavior: Microphone captures audio (1-second windows) -> MFCC computation -> DS-CNN inference -> “stop” detected -> Serial output timestamp string -> Python logging script on PC appends to `distraction_log.csv`

Logging script: A simple Python script reads from the Serial port and writes each detection event to a CSV file with columns: timestamp, date, time, `day_of_week`.

Evaluation:

- Accuracy measured on held-out test split of Speech Commands dataset (10%)
- False positive rate tested in realistic ambient noise (typing, music, conversation)
- Latency measured from speech onset to Serial output (target: under 1 second)

5. Challenges and Solutions

- False positives from similar-sounding words (“shop”, “top”): include diverse negative examples during training and tune detection threshold
- Acoustic mismatch between dataset (studio recordings) and real-world use: supplement with personal recordings in the actual use environment
- Model size exceeding Flash memory limit: apply 8-bit quantization and use compact DS-CNN architecture
- Serial logging reliability: add a simple handshake protocol and buffer missed events
- Low detection confidence in noisy environments: require two consecutive high-confidence predictions before logging

6. Estimated Timeline and Division of Work

This is a solo project. The following timeline is planned across three weeks:

- Week 1: Download Speech Commands dataset; run Lab 4 Colab notebook end-to-end; verify baseline `micro_speech` example works on hardware
- Week 2: Retrain model targeting only stop vs. background; quantize and export TFLite model; deploy to Arduino and test detection accuracy
- Week 3: Write Python Serial logging script; test full pipeline in real use conditions; collect distraction log data; prepare final report and demo

7. Request for Additional Hardware

No additional hardware is requested. This project uses only the Arduino Nano 33 BLE Sense already available, specifically its built-in microphone. All other components (PC for logging, USB cable) are standard and already on hand.